

Algeria History Database

Project Report

Author	Dellali Abdelhamid
Institution	National School of Cybersecurity (NSCS)
Department	Foundation Training
Course	Algorithms and Dynamic Data Structures (ALSDS)
Level	1 st Year
Academic Year	2025/2026
Language	C (C11) with Raylib/Raygui GUI

0 Contents

1	Project Overview	2
2	Linked Lists and Queues	3
2.1	Design	3
2.2	parsePersonalityLine — Single-Pass Line Parser	3
2.3	rewriteFile — Atomic Database Persistence	4
2.4	mergeNodes — Building the Doubly Linked List	4
2.5	ageP — Sort Personalities by Age into a Queue	5
3	Stacks	6
3.1	Design	6
3.2	addNameStack — Sorted Insertion into a Stack	6
3.3	stackToQueue — Cross-Structure Conversion	7
3.4	isPersonalityKilled — Keyword-Based Classification	7
3.5	recRevStack — Recursive Stack Reversal	8
4	Binary Search Tree	8
4.1	Design	8
4.2	deleteNameBST — Three-Case Deletion	8
4.3	BTSMerge — Merge Two Trees into a Balanced BST	9
4.4	lowestCommonAncestor — Exploiting BST Structure	10
4.5	BSTMirror — In-Place Tree Mirroring	10
5	Recursion Module	11
5.1	Design	11
5.2	removeOccurrence — Recursive Line Filtering	11
5.3	namePermutation — Backtracking Permutation	12
5.4	replaceOccurrence — Recursive Targeted Date Update	12
6	Graphical Interface	13
7	Deviations from the Specification	13
	Conclusion	15

1 Project Overview

This is a project I designed to manage a database of Algerian historical personalities, events, and important dates, stored entirely in a plain-text file. The goal was to use this domain as the playground for implementing every major dynamic data structure covered in the ALSDS course: singly linked lists, doubly linked lists, circular lists, queues, stacks, binary search trees, and a purely recursive file-processing module. On top of all of that, I built a graphical user interface using Raylib so the whole thing is actually usable.

The data lives in `data/algeria_history.txt` and follows a simple grammar I designed to be both human-readable and easy to parse with a single-pass scanner:

```
# Comment lines start with #
Emir Abdelkader = Algerian national hero... {1808-09-06} {1883-05-26}
    Personality: NAME = DEFINITION {DoB} {DoD}

Battle of Algiers : 1956-09-30
    Event: NAME : YYYY-MM-DD

{1954-11-01}
    Standalone important date
```

The parser classifies each line by looking at its first non-whitespace character and the presence of `=`, `:`, or `{`, which is enough to unambiguously determine line type. All data structures are typed in `include/types.h` and all modules have clean header/implementation separation.

1.0 Data Structures

Type	Kind	Role
TList	Singly linked list	Personalities (name + definition)
TListDate	Singly linked list	Personalities (name + DoB + DoD)
TListEvent	Singly linked list	Events (name + date)
TBiList	Doubly linked list	Merged personality record
TCircList	Circular linked list	Merged personality record
TStack	Linked stack	Primary in-memory working structure
TQueue	Linked queue (front/rear)	Sorted output pipelines
TTree	Binary Search Tree	Name-keyed fast lookup

1.0 Module layout

```
src/utils.c      Date arithmetic, string helpers, palindrome check
src/file_parser.c
                  File I/O: classifying, parsing, and rewriting
src/list.c       All linked-list and queue operations
src/stack.c      All stack operations
src/tree.c       All BST operations
```

src/recursion.c Purely recursive file and string operations

src/gui.c 941-line Raylib/Raygui graphical interface

2 Linked Lists and Queues

2.1 Design

I keep two parallel singly linked lists in lock-step: `TList` carries names and definitions while `TListDate` carries names and dates. Index i in one always corresponds to index i in the other. This separates concerns cleanly and avoids bloating every node with rarely-used date fields when only text-searching is needed. The queue (`TQueue`) wraps a front/rear pointer pair over `TQNode` elements, giving $O(1)$ enqueue and dequeue.

2.2 parsePersonalityLine — Single-Pass Line Parser

```

1  bool parsePersonalityLine(const char *line,
2                             char *name_out, char *def_out,
3                             Date *dob_out,  Date *dod_out) {
4      const char *eq = strchr(line, '=');
5      if (!eq) return false;
6
7      // Name is everything to the left of '='
8      int name_len = (int)(eq - line);
9      strncpy(name_out, line, name_len);
10     name_out[name_len] = '\0';
11     trimWhitespace(name_out);
12
13     // Work on the right-hand side only
14     char rest[MAX_LINE];
15     strncpy(rest, eq + 1, MAX_LINE - 1);
16
17     // Extract DoB and DoD from first and second {...}
18     char date_buf[MAX_DATE];
19     int pos = 0;
20     if (extractBraces(rest, &pos, date_buf, MAX_DATE))
21         *dob_out = parseDate(date_buf);
22     if (extractBraces(rest, &pos, date_buf, MAX_DATE))
23         *dod_out = parseDate(date_buf);
24
25     // Definition is everything up to the first '{'
26     int brk = 0;
27     while (rest[brk] && rest[brk] != '{') brk++;
28     strncpy(def_out, rest, brk);
29     trimWhitespace(def_out);
30     return name_out[0] != '\0';
31 }
```

Why this is interesting. The parser has to handle a line format where the definition itself may contain colons, dashes, and numbers — the same characters that appear in

dates. Rather than using a regex or a multi-pass approach, I locate the = sign first to split name from payload, then use brace scanning (`extractBraces`) to find dates by their syntactic markers. The definition is then implicitly everything between = and the first {. A single left-to-right scan handles all three fields with no backtracking.

2.3 rewriteFile — Atomic Database Persistence

```

1  bool rewriteFile(const char *filename,
2                  TList *personalities, TListDate *dates,
3                  TListEvent *events) {
4      FILE *f = fopen(filename, "w"); // truncate and rewrite
5      fprintf(f, "# Algeria History Database\n");
6      fprintf(f, "# FORMAT: NAME = DEFINITION {DOB} {DOD} |
... \n\n");
7
8      TList *p = personalities;
9      TListDate *d = dates;
10     while (p && d) {
11         char dob_buf[MAX_DATE], dod_buf[MAX_DATE];
12         dateToString(d->dob, dob_buf);
13         dateToString(d->dod, dod_buf);
14
15         if (!dateIsNull(d->dob) && !dateIsNull(d->dod))
16             fprintf(f, "%s = %s {%s} {%s}\n", p->name,
p->definition, dob_buf, dod_buf);
17         else if (!dateIsNull(d->dob))
18             fprintf(f, "%s = %s {%s}\n", p->name,
p->definition, dob_buf);
19         else
20             fprintf(f, "%s = %s\n", p->name, p->definition);
21
22         p = p->next; d = d->next;
23     }
24     for (TListEvent *ev = events; ev; ev = ev->next) {
25         char date_buf[MAX_DATE];
26         dateToString(ev->date, date_buf);
27         fprintf(f, "%s: %s\n", ev->event_name, date_buf);
28     }
29     fclose(f);
30     return true;
31 }
```

Why this is interesting. Every mutation (add, delete, update) calls `rewriteFile` after updating the in-memory lists, which means the file is always the serialised projection of the live linked lists — no append-and-patch, no seek-and-overwrite. The function intentionally handles missing dates gracefully: if only DoB is known the DoD brace is simply omitted, and `parsePersonalityLine` will re-parse that correctly on the next load. It is the single point of truth for how data is written to disk.

2.4 mergeNodes — Building the Doubly Linked List

```

1  TBiList *mergeNodes(TList *s, TListDate *a) {
2      TBiList *head = NULL, *tail = NULL;
3      TList *p = s;
4      TListDate *d = a;
5      while (p && d) {
6          TBiList *node = (TBiList *)malloc(sizeof(TBiList));
7          strncpy(node->name, p->name, MAX_NAME - 1);
8          strncpy(node->definition, p->definition, MAX_DEF - 1);
9          node->dob = d->dob;
10         node->dod = d->dod;
11         node->prev = tail; // link backward
12         node->next = NULL;
13         if (tail) tail->next = node;
14         else head = node;
15         tail = node;
16         p = p->next; d = d->next;
17     }
18     return head;
19 }

```

Why this is interesting. This is where the two parallel lists are finally unified into a single rich node. The key detail is the `prev` pointer assignment: `node->prev = tail` is set *before* advancing `tail`, so it points at the last fully linked node. The forward link (`tail->next = node`) is set on the *previous* iteration's tail, not the new node, which is the standard tail-insert pattern. The result is a fully consistent doubly linked list built in one linear pass.

2.5 ageP — Sort Personalities by Age into a Queue

```

1  TQueue *ageP(TListDate *a) {
2      int n = 0;
3      for (TListDate *cur = a; cur; cur = cur->next) n++;
4
5      typedef struct { TListDate *node; int age_value; } AgeItem;
6      AgeItem *age_items = malloc(n * sizeof(AgeItem));
7
8      int i = 0;
9      for (TListDate *cur = a; cur; cur = cur->next) {
10         age_items[i].node = cur;
11         age_items[i].age_value = ageFromDates(cur->dob,
12         cur->dod);
13         i++;
14     }
15     qsort(age_items, n, sizeof(AgeItem), compare_age_value);
16
17     TQueue *q = queueCreate();
18     for (int j = 0; j < n; j++)
19         enqueue(q, age_items[j].node->name, "",
20         age_items[j].node->dob,
21         age_items[j].node->dod);

```

```

21     free(age_items);
22     return q;
23 }

```

Why this is interesting. Sorting a linked list in-place efficiently is hard; sorting an array is trivial. I use the flatten-sort-rebuild idiom throughout the project: materialise the list to a temporary array of structs carrying the sort key (here, computed integer age), sort it, then drain it into the output structure. The computed age is stored alongside the pointer so `ageFromDates` is called once per node, not once per comparison. The output is a queue rather than a list because the consuming code (the GUI’s “By Age” tab) only needs to dequeue from front to back.

3 Stacks

3.1 Design

The stack is the *central working structure* of the application. After loading the database, I merge the two parallel lists into a `TBiList`, then convert it to a `TStack`. From there, every other structure (the BST, the sorted queue, the sorted list) is derived from the stack. This pipeline — file → lists → merged → stack → tree/queue — makes the stack the single source of truth for in-memory operations.

3.2 `addNameStack` — Sorted Insertion into a Stack

```

1  TStack *addNameStack(TStack *stk, const char *name,
2                      const char *definition, Date dob, Date
3                      dod) {
4      TStack *node = stackCreate(name, definition, dob, dod);
5      // Insert at front if stack is empty or name sorts before
6      // current top
7      if (!stk || strcasecmp(name, stk->name) <= 0) {
8          node->next = stk;
9          return node;
10     }
11     // Walk until we find the right insertion point
12     TStack *cur = stk;
13     while (cur->next && strcasecmp(name, cur->next->name) > 0)
14         cur = cur->next;
15     node->next = cur->next;
16     cur->next = node;
17     return stk;
18 }

```

Why this is interesting. A stack is normally only pushed to the top, but this function treats it as a sorted linked list and inserts at the lexicographically correct position. The only pointer that can change is either the head (if the new name sorts before everything) or `cur->next` somewhere in the middle. This is $O(n)$ but preserves alphabetical invariant without a separate sort pass, which matters when the GUI’s “Add” operation must keep the view sorted immediately.

3.3 stackToQueue — Cross-Structure Conversion

```

1  TQueue *stackToQueue(TStack *stk) {
2      // Step 1: deep-copy the stack (push reverses order, so
   copy first)
3      TStack *copy = NULL;
4      for (TStack *cur = stk; cur; cur = cur->next)
5          push(&copy, cur->name, cur->definition, cur->dob,
   cur->dod);
6
7      // Step 2: sort the copy alphabetically
8      TStack *sorted = sortNameStack(copy);
9
10     // Step 3: drain sorted stack into a queue (FIFO order =
   alpha order)
11     TQueue *q = queueCreate();
12     while (sorted) {
13         TStack *node = pop(&sorted);
14         enqueue(q, node->name, node->definition, node->dob,
   node->dod);
15         free(node);
16     }
17     return q;
18 }

```

Why this is interesting. Converting a stack to a queue naively would reverse the order (stack is LIFO, queue is FIFO). But the goal here is a *sorted* queue, not a reversed one. So I first copy the stack (which reverses it due to push semantics), sort the copy, then drain it into the queue front-to-back. The copy step is necessary because `sortNameStack` is destructive — it pops and relinks nodes. The result is a queue whose front is the alphabetically smallest name, ready to be dequeued in order.

3.4 isPersonalityKilled — Keyword-Based Classification

```

1  bool isPersonalityKilled(TStack *stk, const char *name) {
2      TStack *node = getInfoPersonality(stk, name);
3      if (!node) return false;
4      char def_lower[MAX_DEF];
5      strToLower(def_lower, node->definition);
6      return strstr(def_lower, "killed")      != NULL ||
7             strstr(def_lower, "martyred")    != NULL ||
8             strstr(def_lower, "executed")    != NULL ||
9             strstr(def_lower, "assassinated") != NULL;
10 }

```

Why this is interesting. This is a simple but semantically rich function. Rather than maintaining a separate boolean field per personality, I derive the classification at query time from the definition text. The case-insensitive keyword list covers four different ways a violent death might be described in historical writing. The approach is extensible (adding a keyword is one line) and keeps the data model clean — no extra field needed in

the struct. It also demonstrates that not every query needs a dedicated data structure; sometimes the definition itself is the data.

3.5 recRevStack — Recursive Stack Reversal

```

1 static void pushBottom(TStack **stk, TStack *node) {
2     if (!*stk) { node->next = NULL; *stk = node; return; }
3     TStack *top = pop(stk);    // temporarily remove top
4     pushBottom(stk, node);    // recurse: insert node at
    bottom of rest
5     top->next = *stk;          // restore top above the
    newly-inserted node
6     *stk = top;
7 }
8
9 TStack *recRevStack(TStack *stk) {
10     if (!stk || !stk->next) return stk;
11     TStack *top = pop(&stk);
12     stk = recRevStack(stk);    // reverse the rest
13     pushBottom(&stk, top);    // send old top to the bottom
14     return stk;
15 }

```

Why this is interesting. This implements stack reversal using only the stack's own primitive operations (push/pop) and no auxiliary structure. The recursion depth of `recRevStack` is $O(n)$; for each frame, `pushBottom` recurses another $O(n)$ levels, giving $O(n^2)$ total calls. It is not the most efficient reversal, but it is the *only* way to do it purely recursively without an extra array or list, which is exactly what the course exercise requires.

4 Binary Search Tree

4.1 Design

The BST uses the personality name as its sort key (case-insensitive). Every node is self-contained: name, definition, DoB, and DoD. The tree is built by inserting the entire stack into an initially empty root, which means its balance depends on insertion order (effectively random with respect to alphabetical order). Explicit rebalancing is available via `BTSMerge`.

4.2 deleteNameBST — Three-Case Deletion

```

1 TTree *deleteNameBST(TTree *tr, const char *name) {
2     if (!tr) return NULL;
3     int cmp = strcasecmp(name, tr->name);
4     if      (cmp < 0) tr->left  = deleteNameBST(tr->left,
    name);
5     else if (cmp > 0) tr->right = deleteNameBST(tr->right,
    name);

```

```

6     else {
7         // Case 1: no left child -> promote right subtree
8         if (!tr->left) { TTree *r = tr->right; free(tr);
return r; }
9         // Case 2: no right child -> promote left subtree
10        if (!tr->right) { TTree *l = tr->left; free(tr);
return l; }
11        // Case 3: two children -> copy in-order successor's
data in,
12        //          then delete the successor from the right
subtree
13        TTree *successor = findMin(tr->right);
14        strncpy(tr->name,          successor->name,
MAX_NAME - 1);
15        strncpy(tr->definition, successor->definition, MAX_DEF
- 1);
16        tr->dob = successor->dob;
17        tr->dod = successor->dod;
18        tr->right = deleteNameBST(tr->right, successor->name);
19    }
20    return tr;
21 }

```

Why this is interesting. The two-children case is the hardest part of BST deletion. Rather than restructuring pointers (which would require updating multiple parent links), I copy the in-order successor's data into the current node and delete the successor from the right subtree. The successor is always the leftmost node of the right subtree, so `findMin` is a simple walk. Because the successor has at most one child (no left child by definition), its deletion always hits case 1 or 2 — no infinite recursion.

4.3 BTSMerge — Merge Two Trees into a Balanced BST

```

1  TTree *BTSMerge(TTree *tr1, TTree *tr2) {
2      int n1 = treeSize(tr1), n2 = treeSize(tr2);
3      TTree **arr1 = malloc(n1 * sizeof(TTree *));
4      TTree **arr2 = malloc(n2 * sizeof(TTree *));
5
6      // In-order traversal = sorted array (BST property)
7      int idx1 = 0, idx2 = 0;
8      collectInOrder(tr1, arr1, &idx1);
9      collectInOrder(tr2, arr2, &idx2);
10
11     // Merge two sorted arrays in O(n+m)
12     int total;
13     TTree **merged = mergeSortedArrays(arr1, n1, arr2, n2,
&total);
14     free(arr1); free(arr2);
15
16     // Build height-balanced BST: always pick midpoint as root
17     TTree *root = buildBalanced(merged, 0, total - 1);
18     free(merged);

```

```

19     return root;
20 }
21
22 static TTree *buildBalanced(TTree **arr, int start, int end) {
23     if (start > end) return NULL;
24     int mid = (start + end) / 2;
25     TTree *root = nodeCreate(arr[mid]->name,
26                               arr[mid]->definition,
27                               arr[mid]->dob,   arr[mid]->dod);
28     root->left  = buildBalanced(arr, start,   mid - 1);
29     root->right = buildBalanced(arr, mid + 1, end);
30     return root;
31 }

```

Why this is interesting. This function exploits the BST invariant twice. First, in-order traversal of a BST is sorted by definition, so `collectInOrder` on each tree yields two sorted arrays for free. Second, building a balanced BST from a sorted array by always choosing the midpoint as root guarantees height $\lfloor \log_2 n \rfloor$. The merge step (standard merge-sort merge) runs in $O(n+m)$. The total cost is $O(n+m)$ — far cheaper than inserting one tree’s nodes into the other one by one, which would be $O(n \log(n+m))$.

4.4 lowestCommonAncestor — Exploiting BST Structure

```

1 TTree *lowestCommonAncestor(TTree *tr,
2                               const char *word1, const char
3                               *word2) {
4     if (!tr) return NULL;
5     int cmp1 = strcasecmp(word1, tr->name);
6     int cmp2 = strcasecmp(word2, tr->name);
7     // Both keys smaller -> LCA is in left subtree
8     if (cmp1 < 0 && cmp2 < 0)
9         return lowestCommonAncestor(tr->left, word1, word2);
10    // Both keys larger -> LCA is in right subtree
11    if (cmp1 > 0 && cmp2 > 0)
12        return lowestCommonAncestor(tr->right, word1, word2);
13    // Keys straddle current node -> this IS the LCA
14    return tr;
15 }

```

Why this is interesting. The LCA algorithm for a general binary tree requires $O(n)$ time and parent-pointer tracking. In a BST it reduces to $O(h)$ with no extra memory, because the ordering invariant tells us exactly which subtree the LCA must be in. The split condition (`cmp1` and `cmp2` have opposite signs, or one of them is zero) is handled implicitly: if we cannot go left and cannot go right, the current node straddles the two keys and is therefore their LCA by definition.

4.5 BSTMirror — In-Place Tree Mirroring

```

1 TTree *BSTMirror(TTree *tr) {
2     if (!tr) return NULL;

```

```

3   TTree *tmp    = tr->left;
4   tr->left      = BSTMirror(tr->right);  // recurse right,
    assign to left
5   tr->right     = BSTMirror(tmp);        // recurse on saved
    left
6   return tr;
7 }

```

Why this is interesting. The elegance here is that the recursion and the swap happen simultaneously. Saving `tr->left` into `tmp` before overwriting it is the only explicit bookkeeping needed. Each node is visited once, and the function returns the same root pointer (the tree is mirrored in-place). After mirroring, the right-to-left traversal of the mirrored tree equals the left-to-right traversal of the original — which reverses the alphabetical order, useful for displaying names in reverse-alphabetical order.

5 Recursion Module

5.1 Design

The recursion module operates directly on the open file, replacing explicit `while` loops with recursive calls. Each recursive helper reads one line per invocation and accumulates its result in the return value. This is a deliberate exercise in thinking recursively about I/O, not just data structures.

5.2 `removeOccurrence` — Recursive Line Filtering

```

1  static void removeRec(FILE *in, FILE *out, const char *word,
2                          char *line, int sz) {
3      if (!fgets(line, sz, in)) return;          // base case: EOF
4      if (!strcasestr(line, word))
5          fputs(line, out);                      // keep line if
    word absent
6      removeRec(in, out, word, line, sz);        // process rest
    of file
7  }
8
9  bool removeOccurrence(const char *filename, const char *word) {
10     char tmp[512];
11     snprintf(tmp, sizeof(tmp), "%s.tmp", filename);
12     FILE *in  = fopen(filename, "r");
13     FILE *out = fopen(tmp, "w");
14     char line[MAX_LINE];
15     removeRec(in, out, word, line, MAX_LINE);
16     fclose(in); fclose(out);
17     remove(filename);
18     rename(tmp, filename);
19     return true;
20 }

```

Why this is interesting. The write-to-temp-then-rename pattern is the correct way to update a file without risking corruption: if the process dies mid-write, the original file is still intact. The recursion replaces the loop entirely — base case is EOF, recursive case is “process the rest”. The filtering logic (fputs only if the word is absent) sits between the read and the recursive call, which is the tail position — meaning the function processes lines in the same order they appear in the file.

5.3 namePermutation — Backtracking Permutation

```

1 static void permuteRec(char *name, int start, int len) {
2     if (start == len - 1) { printf(" %s\n", name); return; }
3     for (int i = start; i < len; i++) {
4         // Fix position 'start' to character name[i]
5         char tmp = name[start]; name[start] = name[i]; name[i]
6         = tmp;
7         permuteRec(name, start + 1, len);
8         // Undo the swap (backtrack)
9         tmp = name[start]; name[start] = name[i]; name[i] =
10        tmp;
11    }
12 }
```

Why this is interesting. This is the canonical in-place backtracking permutation algorithm. The string is mutated in place, the recursive call explores all completions with name[start] fixed, and the swap is undone before the next iteration of the loop. The output is all $n!$ permutations without allocating any extra string buffers. For a name like “Ali” (3 characters) this produces 6 permutations; for “Abdelkader” (10 characters) it would produce 3,628,800 — which is why the GUI only calls this for short inputs.

5.4 replaceOccurrence — Recursive Targeted Date Update

```

1 static void replaceRec(FILE *in, FILE *out, const char *name,
2                        Date dob, Date dod, char *line, int sz)
3 {
4     if (!fgets(line, sz, in)) return;
5     if (strcasestr(line, name) &&
6         classifyLine(line) == LINE_PERSONALITY) {
7         char pname[MAX_NAME], def[MAX_DEF];
8         Date old_dob, old_dod;
9         if (parsePersonalityLine(line, pname, def, &old_dob,
10        &old_dod) &&
11             strcmp(pname, name) == 0) {
12             // Re-emit the line with updated dates
13             char dob_buf[MAX_DATE], dod_buf[MAX_DATE];
14             dateToString(dob, dob_buf);
15             dateToString(dod, dod_buf);
16             fprintf(out, "%s = %s {%s} {%s}\n", pname, def,
17        dob_buf, dod_buf);
18             replaceRec(in, out, name, dob, dod, line, sz);
19         }
20     }
21 }
```

```
17     }
18   }
19   fputs(line, out);
20   replaceRec(in, out, name, dob, dod, line, sz);
21 }
```

Why this is interesting. Unlike `removeOccurrence`, this function cannot simply skip or copy lines — it must surgically reconstruct a personality line with new date values. The double guard (`strcasestr` for a quick pre-filter, then `classifyLine` + full `parsePersonalityLine` + exact name match) prevents false matches on lines that merely mention the name in a definition. The recursive structure handles arbitrary file length without loading the file into memory.

6 Graphical Interface

I built the GUI using **Raylib 6.0** and the single-header **Raygui** extension. The entire interface is driven by a single `App` struct holding all state: the active view, active tab, input fields, a result-line buffer, and the live data pointers.

The layout is a left sidebar of five views (Linked Lists, Queues, Stacks, BST, Recursion), a tab strip at the top of the content area selecting the specific operation within that view, and a scrollable result panel. Context-sensitive input fields (up to 5 per tab) appear above the result panel with labels that change per operation. A “Run” button dispatches to the correct module function, captures its output into the result buffer via `resAdd()`, and refreshes the display.

The BST view additionally renders an interactive tree diagram: nodes are drawn as circles connected by lines, positioned recursively from the live `TTree` pointer, with pan and zoom support via mouse drag and scroll wheel.

The colour scheme is dark navy with Algeria-flag green accents for active elements and red for destructive actions (delete, remove). All data loading happens once at startup via `loadData()`, and any mutation calls `loadData()` again to keep the in-memory state consistent with the file.

7 Deviations from the Specification

The following documents where my implementation differs from the function signatures and descriptions given in the project PDF, with a technical rationale for each decision.

Specification	Implementation	Technical reason
<pre>getPersonality(File *f) and *filename and open/close getDatePersonality(File *f) take an open File*</pre>	Both take const char *filename and open/close file internally	Passing an open file handle forces the caller to manage file state. Using a filename lets each function own its file lifetime, avoids use-after-close bugs, and matches how every other loader in the module works.
<pre>getInfoByDates and getInfoByDates2 take TList *DoB / TList *DoD</pre>	Both take TListDate *DoB / TListDate *DoD	Dates do not live in TList; they live in TListDate. Using TList here would require either casting or storing dates in the text-list nodes, which would break the separation of concerns the dual-list design is built on.
<pre>deletepersonality returns TList*</pre>	Returns bool; takes TList **s and TListDate **a	Returning only the personality list head would leave the caller with no way to update the date list. A bool return with double-pointer arguments lets one call atomically update both lists and report failure, without the caller having to manually sync a second list.
<pre>addPersonality signature has no definition parameter</pre>	Adds const char *definition as a parameter	A personality with no definition is an incomplete record. The file format requires a definition field; omitting it from the add-function signature would mean silently writing empty strings to disk or inventing a separate update step just to fill in the definition.
<pre>addNameBST is specified to return TStack*</pre>	Returns TTree*	This is a clear typo in the specification. A BST insertion function that returns a stack pointer is semantically incoherent. The function is a thin wrapper around <code>treeInsert</code> , which must return the (possibly new) root of the tree.
<pre>countPersonality should match by birth death or</pre>	Delegates to similarPersonality, which only searches definition text	Cross-referencing event dates against personality years requires joining three separate

7 Conclusion

The project implements all required modules: three variants of linked lists, a FIFO queue, a LIFO stack, a binary search tree, and a recursive file-processing module. Each module exposes a coherent API that shares the same domain types from `types.h`. Recurring design patterns — parallel linked lists, the flatten-sort-rebuild idiom for sorting linked structures, and the temp-file-rename pattern for safe persistence — appear consistently across modules.

The Raylib GUI makes the database genuinely interactive: every function is reachable from a tab, inputs are labelled per operation, and results are rendered in a scrollable panel. The BST tab additionally renders a live visual of the tree structure.

The deviations from the spec are driven by type correctness, API design consistency, and removing ambiguities that would produce incorrect or incomplete behaviour. In each case the implementation is a strict improvement over the specified signature.